

Theoretical Computer Science Exam, January 17th, 2019

The exam consists of **4 exercises**. Available time: 1 hr 40 min.; you may write your answers in Italian, if you wish.

All exercises must be addressed for the written exam to be considered sufficient.

F.NAME L.NAME MATRICOLA

I have passed the midterm exam and I intend to skip the first part (Ex.1-2) (select one): **YES NO**

Exercise 1 (8 pts)

Consider the following (partial) function, defined on even-length strings and undefined for odd-length strings.

$$\text{src}(x) = yz^R \text{ if and only if } x = yz \text{ and } |y| = |z|$$

Thus, for instance, $\text{src}(abcd) = abdc$, $\text{src}(abbabb) = abbbba$, $\text{src}(\varepsilon) = \varepsilon$.

The src function is also generalized by applying it to languages, with the following, customary definition.

$$\text{src}(L) = \{ \text{src}(x) \mid x \in L \}$$

In words, $\text{src}(L)$ is defined as the language that contains exactly the strings obtained by applying the src function to a string of L .

Next, consider the grammar G , with axiom S and alphabet $\{a, b\}$ defined by the following rules.

$$S \rightarrow aPa \mid \varepsilon$$

$$P \rightarrow bSb$$

1. Write the four shortest strings of $L(G)$, in order of increasing length.
2. Write a concise, formal and precise definition of $L(G)$.
3. Write the result of applying function src to the four shortest strings of $L(G)$ found in the previous question.
4. Write a concise, formal and precise definition of $\text{src}(L(G))$, and determine which is the least powerful type of automaton that accepts $\text{src}(L(G))$.
5. Write a context-free grammar that generates $\text{src}(L(G))$.
6. Determine if the family of regular languages is closed under the src function. Provide a justification for your answer.

Exercise 2 (8 pts)

Consider the two language expressions

$$L_1 = ((a|b)(c|d))^+ \text{ and } L_2 = ((a|b)(b|c))^+$$

Notice that the two languages are not disjointed and none of the two is included in the other; in fact, the following string inclusions hold (where the operator '-' here denotes set difference):

$$x_1 = ad \in L_1 - L_2, \quad x_{12} = ac \in L_1 \cap L_2, \quad x_2 = ab \in L_2 - L_1.$$

1. Write a monadic logic sentence φ_1 (first-order, or second-order only if necessary) such that $L_1 = L(\varphi_1)$. Verify that $x_2 = ab \notin L(\varphi_1)$ and explain in logical terms the reason why $ab \neq \varphi_1$.
2. Write a monadic logic sentence φ_2 (first-order, or second-order only if necessary) such that $L_2 = L(\varphi_2)$. Verify that $x_1 = ad \notin L(\varphi_2)$ and $y = abb \notin L(\varphi_2)$ and explain in logical terms the reason why $ad \neq \varphi_2$ and $abb \neq \varphi_2$.

Exercise 3 (8 pts)

We define a *monotonic grammar* as an unrestricted grammar that satisfies the following constraints for every rule of the form $\alpha \rightarrow \beta$:

- $\alpha \in V_N^+$ where V_N is the set of nonterminal symbols
- $\beta \in (V_N \cup V_T)^+$ where V_T is the set of terminal symbols
- $|\alpha| \leq |\beta|$

Notice that the notions of derivation and language generated by the grammar are defined as usual.

Answer the following questions as precisely and clearly as possible, providing suitable justifications.

1. Is the language generated by a monotonic grammar decidable? Is it semidecidable?
2. What changes concerning the decidability and semidecidability of the generated language if the above constraints are removed?

Exercise 4 (8 pts)

The following code fragment, written in a high-level programming language pseudo code, reads two positive integer values into variables n and k , and a sequence of n integer values that are stored in the array a . The program outputs 1 if there is a pair of values stored in array a such that their sum is equal to k , otherwise it outputs 0.

```
read(n);
read(k);
for i from 1 to n do read(a[i]); endfor;
for i from 1 to n do
    for j from 1 to n do
        if a[i]+a[j] = k then
            write(1); halt;
        endif;
    endfor;
endfor;
write(0);
halt;
```

Assume that the above fragment is translated into RAM machine code and executed.

1. Evaluate the time and space complexity class of its execution, as a function of the input size, i.e., of the number of integer values that are read from the input. Use the constant cost criterion.
2. What changes in the above complexity evaluations if the logarithmic cost criterion is adopted? Do the complexity figures change significantly?

Solutions

Exercise 1

1. Write the four shortest strings of $L(G)$, in order of increasing length.

$\varepsilon, abba, ababbaba, abababbababa.$

2. Write a concise, formal and precise definition of $L(G)$.

$$L(G) = \{ (ab)^n (ba)^n \mid n \geq 0 \}$$

3. Write the result of applying function src to the four shortest strings of $L(G)$ found in the previous question.

$\varepsilon, abab, abababab, abababababab.$

4. Write a concise, formal and precise definition of $src(L(G))$.

$$src(L(G)) = \{ (ab)^{2n} \mid n \geq 0 \} \quad \text{the language is regular.}$$

5. Write a context-free grammar that generates $src(L(G))$.

The following non-regular, context-free grammar is obtained by modifying G ; the language obviously admits a regular grammar.

$$S \rightarrow a P b \mid \varepsilon$$

$$P \rightarrow b S a$$

6. Determine if the family of regular languages is closed under the src function. Provide a justification for your answer.

The family of regular languages is *not* closed under the src function. Consider the regular language $L = \{ (ab)^{2n} \mid n \geq 0 \}$. Then $src(L) = \{ (ab)^n (ba)^n \mid n \geq 0 \}$ is not a regular language.

Exercise 2

φ_1 such that $L_1 = L(\varphi_1)$

$$\forall x ((first(x) \rightarrow a(x) \vee b(x)) \wedge$$

$$(last(x) \rightarrow c(x) \vee d(x)) \wedge$$

$$((a(x) \vee b(x)) \rightarrow c(x+1) \vee d(x+1)) \wedge$$

$$((c(x) \vee d(x)) \wedge \neg last(x) \rightarrow a(x+1) \vee b(x+1))$$

)

$x_2 = ab \notin L(\varphi_1)$ because for this string $b(1)$ holds, therefore for $x=1$ the subformula $(last(x) \rightarrow c(x) \vee d(x))$ evaluates to false.

φ_2 such that $L_2 = L(\varphi_2)$

$$\exists X \forall x (\neg X(0) \wedge (\neg last(x) \rightarrow (X(x) \leftrightarrow \neg X(x+1))) \wedge$$

$$(\neg X(x) \rightarrow a(x) \vee b(x)) \wedge$$

$$(X(x) \rightarrow b(x) \vee c(x)) \wedge$$

$$(last(x) \rightarrow X(x))$$

)

$x_1 = ad \notin L(\varphi_2)$ because for this string $d(1)$ holds, therefore for $x=1$ the subformula $(X(x) \rightarrow b(x) \vee c(x))$ evaluates to false.

$y = abb \notin L(\varphi_2)$ because $|y|=3$, therefore for $x=2$ the subformula $(last(x) \rightarrow X(x))$ evaluates to false.

Since the two languages do not include ε , one must add the standard clause for ruling out the empty string ε :

$$\exists x (a(x) \vee \neg a(x))$$

Exercise 3

1. The language generated by a monotonic grammar is decidable. A nondeterministic Turing machine that, having as input a string x , decides if it belongs to the language, works as follows. It nondeterministically explores all derivations, starting from the axiom S . For every obtained string in $(V_N \cup V_T)^+$ it nondeterministically executes all possible derivation steps, while it keeps track, along every computation path representing a derivation, of all the strings generated so far.
If a string (in $(V_N \cup V_T)^+$) obtained by means of a derivation step has already been generated previously in the same computation path, then the generation terminates with rejection (NB: only for that computation path); if a terminal string is obtained then the generation terminates, and x is accepted if and only if it is equal to the generated string. Notice that the check to rule out that a string is obtained twice in the same derivation is necessary because the length of successively derived strings does not necessarily increase at every step.
2. If the constraints are removed then the grammar is a generic unrestricted grammar, whose generated language is notoriously semidecidable but undecidable.

Exercise 4

1. A total of n^2 sum and comparison operations are performed in the inner loop body (by the $a[i]+a[j]=k$ expression), plus a total of n^2 increment operations of the inner loop counter variable j . Hence the time complexity is $\Theta(n^2)$. The space complexity is $\Theta(n)$.
2. Adopting the logarithmic cost criterion does not affect the complexity evaluation for the “ $a[i]+a[j]=k$ ” expression. It affects, instead, the complexity of the increment operations for the counter variables i and j of the for loops. The dominating term is the one related with the inner loop variable j , which is incremented from 1 to n for n times. The relevant part of the time complexity term concerning these increment operations is

$$n \cdot \sum_{i=1}^n l(i) \in \Theta(n \cdot n \cdot \log(n)) = \Theta(n^2 \cdot \log(n))$$

The space complexity is $3l(n)+1+n \in \Theta(n)$, therefore the complexity class is unchanged.

In conclusion, neither the time and the space complexity are changed in a significant way.